

I'm doing a lab on a Pygame marble tilt maze. Project structure:
main.py, game/game_engine.py, game/marble.py, game/wall.py.

Current behavior: the marble accelerates toward the mouse cursor relative to screen center (simulating a tilt), has friction and a capped top speed, bounces off rectangular walls, there's a goal circle in the far corner and a countdown timer.

I will give you 4 tasks, one at a time. Rules for your answers:

- Give me COMPLETE updated files (or complete functions with the exact file name), so I can paste them in directly.
- Keep the existing class structure and naming style.
- After the code, give me a 2-line "how to test this" note.

Here are the files:

```
--- main.py ---  
--- game/game_engine.py ---  
--- game/marble.py ---  
--- game/wall.py ---
```

Confirm you've read them and briefly describe how collision currently works.

I've read all four files. Here's how collision currently works.

Collision detection: In `_resolve_wall_collisions`, each frame (after the marble moves) the code loops over every wall. It builds the marble's square bounding box with `Marble.rect()` and the wall's rect with `Wall.rect()`, then tests them with `pygame.Rect.colliderect`.

Collision response: If the rects overlap, it computes the overlap depth on each axis and resolves along the axis of *least* penetration:

- If `overlap_x < overlap_y`, it pushes the marble out horizontally (left or right depending on which side of the wall's center it's on) and sets `vx *= -0.3`.
- Otherwise it pushes the marble out vertically and sets `vy *= -0.3`.

The `-0.3` gives a damped bounce, so the marble loses most of its energy on impact.

The known flaw (your Task 1): The test uses the marble's *square* bounding box, not its circle. Near a wall's corner, the square's corner can overlap the wall while the round marble is still visibly clear of it, which produces a "phantom" bounce off empty space. A proper circle-vs-rectangle test (clamp the circle's center to the rect to find the closest point, then compare that distance to the radius) would fix it.

A few other things I noticed that may matter for later tasks:

- **Import paths:** main.py imports game.game_engine, and game_engine.py uses relative imports (.marble, .wall), so the game/ folder needs to be a package and you run from the project root.
- **Timer:** start_ticks is set at construction, so the clock starts at engine creation rather than at first input, and there's no pause or restart logic yet.
- **Game-over state:** It currently just prints once, and there's no on-screen end screen (your Task 2).
- **Input:** handle_event is a stub, so there's no keyboard handling (e.g. restart) yet.

Send over Task 1 whenever you're ready.

Task 1: Fix collision accuracy. Bug: near a wall's corner, the marble bounces off empty space before it visually touches the wall.

Replace the collision logic with proper circle-vs-rectangle collision:

1. Find the closest point on the wall rectangle to the marble's center by clamping the center's x to [rect.left, rect.right] and y to [rect.top, rect.bottom].
2. If the distance from the center to that point is less than the marble radius, it's a collision.
3. Resolve it by pushing the marble out along the normal (center minus closest point) so it no longer overlaps, then reflect the velocity along that normal with a damping factor (e.g., 0.6-0.8).
4. Handle the edge case where the center is exactly inside the rect (distance 0) without a divide-by-zero.
5. Corners must behave like a rounded contact point, not a flat side.

Give me the updated code.

Fixing wall collision handling in the game engine.

Fixing wall collision handling in the game engine.

Only one method changes: replace _resolve_wall_collisions in game/game_engine.py. Nothing else in the file needs to change.

File: game/game_engine.py

python

```
def _resolve_wall_collisions(self):
    m = self.marble

    restitution = 0.7 # how much normal velocity survives a bounce (0.6-0.8 works well)
```

```

for wall in self.walls:

    wall_rect = wall.rect()

    # 1. Closest point on the wall rect to the marble's center
    closest_x = max(wall_rect.left, min(m.x, wall_rect.right))
    closest_y = max(wall_rect.top, min(m.y, wall_rect.bottom))

    # Vector from that point to the marble's center
    nx = m.x - closest_x
    ny = m.y - closest_y
    dist_sq = nx * nx + ny * ny

    # 2. No collision if the closest point is at least one radius away
    if dist_sq >= m.radius * m.radius:
        continue

    if dist_sq > 0:
        # Normal case: center is outside the rect (faces AND corners).
        # Normalizing (center - closest point) gives a flat normal on
        # a side and a diagonal, rounded normal at a corner.
        dist = dist_sq ** 0.5
        nx /= dist
        ny /= dist
        penetration = m.radius - dist
    else:
        # 4. Edge case: center is exactly on or inside the rect, so the
        # closest point IS the center (distance 0, no usable normal).

```

```

# Push out through the nearest face instead.

d_left = m.x - wall_rect.left
d_right = wall_rect.right - m.x
d_top = m.y - wall_rect.top
d_bottom = wall_rect.bottom - m.y
smallest = min(d_left, d_right, d_top, d_bottom)

if smallest == d_left:
    nx, ny = -1.0, 0.0
elif smallest == d_right:
    nx, ny = 1.0, 0.0
elif smallest == d_top:
    nx, ny = 0.0, -1.0
else:
    nx, ny = 0.0, 1.0

# Must travel to the face, then one more radius to clear it
penetration = smallest + m.radius

# 3a. Push the marble out along the normal so it no longer overlaps
m.x += nx * penetration
m.y += ny * penetration

# 3b. Reflect velocity along the normal with damping.
# Only if moving into the wall ( $v \cdot n < 0$ ), so a marble already
# moving away is not pulled back.
vn = m.vx * nx + m.vy * ny
if vn < 0:

```

$$m.vx -= (1 + \text{restitution}) * vn * nx$$
$$m.vy -= (1 + \text{restitution}) * vn * ny$$

How it works:

- **Reflection:** $v - (1 + e)(v \cdot n)n$ flips and scales only the normal component of the velocity by $e = 0.7$. The tangential component is untouched, so the marble slides along walls and curves around corners instead of stopping dead, as the old per-axis - 0.3 did.
- **Corners:** At a corner the closest point is the corner itself, so the normal points diagonally from it. The marble therefore rolls off a rounded contact point and never hits a phantom flat side.
- **Unused code:** `Marble.rect()` is no longer used by the engine. I left it in place, so `marble.py` and `wall.py` don't change.

How to test this:

1. Run the game and steer the marble so it grazes the tips of the internal walls. It should pass with no early bounce until it visibly touches, then deflect diagonally around the corner.
2. Push it hard into a flat wall and into a wall-to-border corner. It should bounce back at about 70% speed, with no jitter, sticking, or passing through walls.

Designing tone generation and bounce-sound trigger logic.

Only `game/game_engine.py` changes. `main.py` already calls `pygame.init()`, which normally starts the mixer, and the new code initializes it itself if that didn't happen. `marble.py` and `wall.py` are untouched. The Task 1 to 3 logic is kept, with small hooks added for sound.

File: `game/game_engine.py`

```
python
```

```
import array
```

```
import math
```

```
import pygame
```

```
from .marble import Marble
```

```
from .wall import Wall
```

```
# Game Engine
```

```
WHITE = (255, 255, 255)
```

```
DARK = (40, 40, 50)
```

```
WALL_COLOR = (90, 90, 110)
```

```
GOAL_COLOR = (60, 200, 120)
```

```
LOSE_COLOR = (230, 90, 90)
```

```
ACCENT_COLOR = (240, 200, 90)
```

```
# Game states
```

```
PLAYING = "PLAYING"
```

```
GAME_OVER = "GAME_OVER"
```

```
MENU = "MENU"
```

```
# Difficulty settings ("time_limit" is in seconds).
```

```
# Medium matches the original values: tilt 0.6, friction 0.02, 45 s.
```

```
DIFFICULTY_SETTINGS = {
```

```
    "Easy": {"tilt": 0.8, "friction": 0.01, "time_limit": 60},
```

```
    "Medium": {"tilt": 0.6, "friction": 0.02, "time_limit": 45},
```

```
    "Hard": {"tilt": 0.4, "friction": 0.05, "time_limit": 30},
```

```
}
```

```
# Menu key -> difficulty name
```

```
DIFFICULTY_KEYS = {
```

```
    pygame.K_1: "Easy",
```

```
    pygame.K_2: "Medium",
```

```
    pygame.K_3: "Hard",
```

```
}
```

```
# Sound tuning

BOUNCE_COOLDOWN_MS = 100 # minimum gap between bounce sounds

MIN_BOUNCE_SPEED = 1.5 # impact speed (along the wall normal) needed for a sound

SAMPLE_RATE = 22050 # used only if we have to initialize the mixer ourselves
```

```
class GameEngine:
```

```
    def __init__(self, width, height):
```

```
        self.width = width
```

```
        self.height = height
```

```
        self.max_speed = 9
```

```
        self.walls = self._build_maze()
```

```
        self.goal_x, self.goal_y, self.goal_radius = width - 60, height - 60, 22
```

```
        self.font = pygame.font.SysFont("Arial", 26)
```

```
        self.title_font = pygame.font.SysFont("Arial", 64, bold=True)
```

```
        self.small_font = pygame.font.SysFont("Arial", 24)
```

```
        # Semi-transparent dark overlay, built once and reused
```

```
        self.overlay = pygame.Surface((width, height), pygame.SRCALPHA)
```

```
        self.overlay.fill((0, 0, 0, 170))
```

```
        # Sound: if anything goes wrong, audio_ok stays False and the
```

```
        # game simply runs silently.
```

```
        self.audio_ok = False
```

```
        self.sounds = {}
```

```
        self._last_bounce_ticks = -BOUNCE_COOLDOWN_MS
```

```
self._init_audio()
```

```
# Start on Medium (original behavior). Sets tilt, friction, time
```

```
# limit, then resets marble, state and timing.
```

```
self.difficulty = "Medium"
```

```
self.start_round("Medium")
```

```
# -----
```

```
# Audio
```

```
# -----
```

```
def _init_audio(self):
```

```
    """Initialize the mixer and build all sounds. Never raises."""
```

```
    try:
```

```
        if not pygame.mixer.get_init():
```

```
            pygame.mixer.init(SAMPLE_RATE, -16, 1)
```

```
    freq, fmt, channels = pygame.mixer.get_init()
```

```
    if fmt != -16:
```

```
        # We generate signed 16-bit samples, so other formats
```

```
        # are not supported here: stay silent instead.
```

```
        raise pygame.error("unsupported mixer sample format")
```

```
# Each segment is (start_freq_hz, end_freq_hz, duration_ms, volume)
```

```
self.sounds = {
```

```
    # short, sharp "tick"
```

```
    "bounce": self._make_sound(
```

```
        [(1200, 900, 40, 0.35)], freq, channels),
```

```
    # rising two-tone: C5 then G5
```

```

        "win": self._make_sound(
            [(523, 523, 130, 0.5), (784, 784, 260, 0.5)], freq, channels),
        # low, descending tone
        "timeout": self._make_sound(
            [(330, 110, 600, 0.5)], freq, channels),
    }

    self.audio_ok = True
except Exception:
    self.audio_ok = False
    self.sounds = {}

```

```

def _make_sound(self, segments, freq, channels):

```

```

    """Build a pygame Sound from (f_start, f_end, ms, volume) segments.

```

```

    Uses only `array` and `math`. Each segment is a sine sweep from
    f_start to f_end with a short attack (avoids clicks) and a decay.

```

```

    """

```

```

    samples = array.array("h") # signed 16-bit

```

```

    for f_start, f_end, ms, volume in segments:

```

```

        n = int(freq * ms / 1000)

```

```

        attack = max(1, int(freq * 0.005)) # 5 ms fade-in

```

```

        phase = 0.0

```

```

        for i in range(n):

```

```

            t = i / n

```

```

            f = f_start + (f_end - f_start) * t

```

```

            phase += 2 * math.pi * f / freq

```

```

            envelope = min(1.0, i / attack) * (1 - t) ** 1.5

```

```

            value = int(32767 * volume * envelope * math.sin(phase))

```

```

        for _ in range(channels): # same sample on every channel
            samples.append(value)

    return pygame.mixer.Sound(buffer=samples.tobytes())

def _play(self, name):
    if not self.audio_ok:
        return

    try:
        self.sounds[name].play()
    except Exception:
        pass # never let audio problems crash the game

def _play_bounce(self):
    """Play the bounce tick, at most once per BOUNCE_COOLDOWN_MS."""
    now = pygame.time.get_ticks()
    if now - self._last_bounce_ticks >= BOUNCE_COOLDOWN_MS:
        self._last_bounce_ticks = now
        self._play("bounce")

# -----
# Rounds and state
# -----

def start_round(self, difficulty):
    """Apply a difficulty's settings and start a fresh round."""
    settings = DIFFICULTY_SETTINGS[difficulty]
    self.difficulty = difficulty
    self.tilt_strength = settings["tilt"]
    self.friction = settings["friction"]

```

```
self.time_limit_ms = settings["time_limit"] * 1000  
self.reset()
```

```
def reset(self):
```

```
    """Fully reset marble, timer and state using the current difficulty."""  
    self.marble = Marble(50, 50) # new marble: start position, zero velocity  
    self.state = PLAYING  
    self.result = None # "solved" or "timeout"  
    self.finish_time_ms = None  
    self.elapsed_ms = 0  
    self.start_ticks = pygame.time.get_ticks()
```

```
def _build_maze(self):
```

```
    walls = []  
    t = 16 # wall thickness  
  
    # outer boundary  
    walls.append(Wall(0, 0, self.width, t))  
    walls.append(Wall(0, self.height - t, self.width, t))  
    walls.append(Wall(0, 0, t, self.height))  
    walls.append(Wall(self.width - t, 0, t, self.height))  
  
    # a few internal walls forming a simple winding path  
    walls.append(Wall(0, 140, self.width - 140, t))  
    walls.append(Wall(140, 260, self.width - 140, t))  
    walls.append(Wall(0, 380, self.width - 140, t))  
  
    return walls
```

```

def _end_game(self, result):
    """Switch to GAME_OVER. elapsed_ms is frozen from here on.

    update() returns early unless PLAYING, so this runs exactly once
    per round, which makes it the right place for the win/lose sound.
    """
    self.state = GAME_OVER
    self.result = result
    if result == "solved":
        self.finish_time_ms = self.elapsed_ms
        self._play("win")
    else:
        self._play("timeout")

def handle_event(self, event):
    # Movement is driven by the continuous mouse position (handle_input).
    # Here we only react to key presses on the end screens.
    # QUIT from the window's X button is handled in main.py in every state.
    if event.type != pygame.KEYDOWN:
        return

    if self.state == GAME_OVER:
        # Any key moves from the result screen to the difficulty menu
        self.state = MENU

    elif self.state == MENU:
        if event.key in DIFFICULTY_KEYS:

```

```

        self.start_round(DIFFICULTY_KEYS[event.key])

    elif event.key in (pygame.K_q, pygame.K_ESCAPE):
        # main.py's loop sees QUIT and exits cleanly
        pygame.event.post(pygame.event.Event(pygame.QUIT))

def handle_input(self):
    if self.state != PLAYING:
        return

    mouse_x, mouse_y = pygame.mouse.get_pos()
    dx = mouse_x - self.width // 2
    dy = mouse_y - self.height // 2
    dist = max(1, (dx ** 2 + dy ** 2) ** 0.5)
    ax = (dx / dist) * self.tilt_strength
    ay = (dy / dist) * self.tilt_strength
    self.marble.vx += ax
    self.marble.vy += ay

def update(self):
    if self.state != PLAYING:
        return

    # Elapsed time only advances while PLAYING, so it freezes on game over
    self.elapsed_ms = pygame.time.get_ticks() - self.start_ticks
    if self.elapsed_ms >= self.time_limit_ms:
        self.elapsed_ms = self.time_limit_ms
        self._end_game("timeout")
    return

```

```
self.marble.vx *= (1 - self.friction)
```

```
self.marble.vy *= (1 - self.friction)
```

```
speed = (self.marble.vx ** 2 + self.marble.vy ** 2) ** 0.5
```

```
if speed > self.max_speed:
```

```
    scale = self.max_speed / speed
```

```
    self.marble.vx *= scale
```

```
    self.marble.vy *= scale
```

```
self.marble.x += self.marble.vx
```

```
self.marble.y += self.marble.vy
```

```
self._resolve_wall_collisions()
```

```
gx = self.goal_x - self.marble.x
```

```
gy = self.goal_y - self.marble.y
```

```
if (gx ** 2 + gy ** 2) ** 0.5 <= self.goal_radius:
```

```
    self._end_game("solved")
```

```
def _resolve_wall_collisions(self):
```

```
    m = self.marble
```

```
    restitution = 0.7 # how much normal velocity survives a bounce (0.6-0.8 works well)
```

```
    for wall in self.walls:
```

```
        wall_rect = wall.rect()
```

```
        # 1. Closest point on the wall rect to the marble's center
```

```

closest_x = max(wall_rect.left, min(m.x, wall_rect.right))
closest_y = max(wall_rect.top, min(m.y, wall_rect.bottom))

# Vector from that point to the marble's center
nx = m.x - closest_x
ny = m.y - closest_y
dist_sq = nx * nx + ny * ny

# 2. No collision if the closest point is at least one radius away
if dist_sq >= m.radius * m.radius:
    continue

if dist_sq > 0:
    # Normal case: center is outside the rect (faces AND corners).
    # Normalizing (center - closest point) gives a flat normal on
    # a side and a diagonal, rounded normal at a corner.
    dist = dist_sq ** 0.5
    nx /= dist
    ny /= dist
    penetration = m.radius - dist
else:
    # 4. Edge case: center is exactly on or inside the rect, so the
    # closest point IS the center (distance 0, no usable normal).
    # Push out through the nearest face instead.
    d_left = m.x - wall_rect.left
    d_right = wall_rect.right - m.x
    d_top = m.y - wall_rect.top
    d_bottom = wall_rect.bottom - m.y

```

```
smallest = min(d_left, d_right, d_top, d_bottom)
```

```
if smallest == d_left:
```

```
    nx, ny = -1.0, 0.0
```

```
elif smallest == d_right:
```

```
    nx, ny = 1.0, 0.0
```

```
elif smallest == d_top:
```

```
    nx, ny = 0.0, -1.0
```

```
else:
```

```
    nx, ny = 0.0, 1.0
```

```
# Must travel to the face, then one more radius to clear it
```

```
penetration = smallest + m.radius
```

```
# 3a. Push the marble out along the normal so it no longer overlaps
```

```
m.x += nx * penetration
```

```
m.y += ny * penetration
```

```
# 3b. Reflect velocity along the normal with damping.
```

```
# Only if moving into the wall ( $v \cdot n < 0$ ), so a marble already
```

```
# moving away is not pulled back.
```

```
vn = m.vx * nx + m.vy * ny
```

```
if vn < 0:
```

```
    # Impact speed is measured BEFORE the reflection changes vn.
```

```
    # Gentle contact (marble resting or sliding along a wall)
```

```
    # stays below the threshold and stays silent.
```

```
    if -vn >= MIN_BOUNCE_SPEED:
```

```
        self._play_bounce()
```

```
m.vx -= (1 + restitution) * vn * nx
```

```
m.vy -= (1 + restitution) * vn * ny
```

```
def _draw_game_over(self, screen):
```

```
    screen.blit(self.overlay, (0, 0))
```

```
    cx, cy = self.width // 2, self.height // 2
```

```
    if self.result == "solved":
```

```
        title = self.title_font.render("YOU WIN!", True, GOAL_COLOR)
```

```
        detail = self.font.render(f"Finish time: {self.finish_time_ms / 1000:.2f}s", True, WHITE)
```

```
    else:
```

```
        title = self.title_font.render("TIME'S UP!", True, LOSE_COLOR)
```

```
        detail = None
```

```
    prompt = self.small_font.render("Press any key to continue", True, WHITE)
```

```
    screen.blit(title, title.get_rect(center=(cx, cy - 50)))
```

```
    if detail is not None:
```

```
        screen.blit(detail, detail.get_rect(center=(cx, cy + 10)))
```

```
    screen.blit(prompt, prompt.get_rect(center=(cx, cy + 60)))
```

```
def _draw_menu(self, screen):
```

```
    screen.blit(self.overlay, (0, 0))
```

```
    cx, cy = self.width // 2, self.height // 2
```

```

title = self.title_font.render("PLAY AGAIN?", True, WHITE)
screen.blit(title, title.get_rect(center=(cx, cy - 110)))

lines = []
for key_label, name in (("1", "Easy"), ("2", "Medium"), ("3", "Hard")):
    secs = DIFFICULTY_SETTINGS[name]["time_limit"]
    lines.append((f"{key_label} - {name} ({secs}s)", WHITE))
lines.append(("Q / Esc - Quit", LOSE_COLOR))

y = cy - 35
for text, color in lines:
    surf = self.font.render(text, True, color)
    screen.blit(surf, surf.get_rect(center=(cx, y)))
    y += 38

def render(self, screen):
    screen.fill(DARK)

    for wall in self.walls:
        pygame.draw.rect(screen, WALL_COLOR, wall.rect())

    pygame.draw.circle(screen, GOAL_COLOR, (self.goal_x, self.goal_y), self.goal_radius)
    pygame.draw.circle(screen, WHITE, (int(self.marble.x), int(self.marble.y)),
self.marble.radius)

# HUD: timer uses the frozen elapsed_ms, so it stops when the game ends
seconds_left = max(0, (self.time_limit_ms - self.elapsed_ms) // 1000)
timer_text = self.font.render(f"Time: {seconds_left}s", True, WHITE)

```

```
screen.blit(timer_text, (10, 10))
```

```
# Current difficulty, right next to the timer
```

```
diff_text = self.font.render(f"[{self.difficulty}]", True, ACCENT_COLOR)
```

```
screen.blit(diff_text, (10 + timer_text.get_width() + 16, 10))
```

```
if self.state == GAME_OVER:
```

```
    self._draw_game_over(screen)
```

```
elif self.state == MENU:
```

```
    self._draw_menu(screen)
```

What changed and why:

- **Generated tones:** `_make_sound` builds signed 16-bit samples in an `array.array("h")` using `math.sin`, then passes `samples.tobytes()` to `pygame.mixer.Sound(buffer=...)`. Each tone is a sine sweep with a 5 ms fade-in to avoid clicks and a decay. The bounce is a 40 ms tick, the win is a rising C5 then G5 pair, and the time-out is a 600 ms sweep from 330 Hz down to 110 Hz.
- **Safe init:** `_init_audio` uses the mixer that `pygame.init()` already started, or starts one itself. It reads the actual frequency and channel count back with `get_init()`, so the buffers match whatever the mixer is using. Any failure, including an unsupported sample format, is caught, `audio_ok` stays False, and `_play` becomes a no-op. The game runs silently.
- **Bounce filtering:** The impact speed `-vn` is measured before the reflection and must be at least `MIN_BOUNCE_SPEED` (1.5). A marble resting on or sliding along a wall only gains about 0.4 to 0.8 of speed per frame from tilt, so it stays silent. `_play_bounce` also enforces the 100 ms cooldown, which covers a marble touching two walls in one frame, such as a corner.
- **Once-only win/lose sound:** The sounds play inside `_end_game`, which runs only on the PLAYING to GAME_OVER transition. `update()` returns early in every other state, so they can't repeat each frame.

How to test this:

1. Slam the marble into a wall for a short "tick" and gently roll along a wall for silence. Reach the goal for the rising two-tone, and run out the clock in another round for the low descending tone, each playing once while the overlay stays up.

2. Tweak `MIN_BOUNCE_SPEED` or `BOUNCE_COOLDOWN_MS` to feel the filtering. To check the silent fallback, temporarily add `raise RuntimeError` at the top of the try in `_init_audio` and confirm the game still runs with no sound.

Now Explain what are those extra things you added

Cataloguing extras added across each separate task.

Here are the things I added beyond your literal spec, grouped by task and with the reason for each. Everything is in `game/game_engine.py`, and I can strip out any of it if your lab wants a leaner version.

Task 1 (collision)

- **`vn < 0` check before reflecting velocity.** You asked for a reflection, but without this guard a marble that is already moving away from the wall would get pulled back in. It also stops the marble from sticking to walls.
- **Nearest-face fallback for the "center inside the rect" case.** You asked me to handle distance 0 without a divide-by-zero. I went a step further and pick a sensible push-out direction instead of just skipping the collision.
- **restitution as a local variable.** This is a named constant (0.7) so you can tune the bounce in one place.

Task 2 (game over)

- **`reset()` method.** I moved marble creation and timing out of `__init__` into `reset()`. You didn't ask for a restart, but "press any key" needs to do something, and Task 3 builds on it.
- **`_end_game()` helper.** It is one place where the transition to `GAME_OVER` happens, so the result and finish time are set exactly once.
- **Clamping `elapsed_ms` on timeout.** This keeps the frozen timer at exactly the limit instead of a few milliseconds over.
- **Cached `SRCALPHA` overlay and extra fonts/colors.** The overlay surface is built once instead of every frame. `title_font`, `small_font` and `LOSE_COLOR` are there for the end screen's look.
- **Removed the old print logic and `_game_over_logged`.** You asked for no console output, so this follows from that.

Task 3 (difficulty)

- **A third state, `MENU`.** You asked for `PLAYING` and `GAME_OVER` in Task 2, but the flow "result screen, then menu" is easiest with a separate state. The result screen now says "Press any key to continue" instead of restarting directly.

- **start_round(difficulty).** It applies the settings, then calls reset(). Tilt, friction and time limit moved out of __init__ because they now depend on the difficulty.
- **DIFFICULTY_KEYS dict.** It maps the 1/2/3 keys to difficulty names, so handle_event doesn't need an if/elif chain.
- **Time limit shown in the menu, ACCENT_COLOR for the HUD label, "PLAY AGAIN?" title.** These are small UI touches.
- **My number choices.** Easy is 0.8 / 0.01 / 60 s and Hard is 0.4 / 0.05 / 30 s. I picked them so Medium stays identical to your original behavior and Hard feels heavy rather than just short.
- **Q/Esc posts pygame.QUIT.** This reuses main.py's existing exit path instead of calling pygame.quit() from inside the engine.

Task 4 (sound)

- **Sine sweeps with fade-in and decay.** You asked for different pitches and lengths. The tones also glide in frequency, and the 5 ms fade-in and decay avoid clicks and harsh cut-offs.
- **_make_sound, _play and _play_bounce helpers.** They keep the try/except and cooldown logic out of the game code.
- **Format and channel handling.** I read the real frequency and channel count back from pygame.mixer.get_init(). If the mixer isn't signed 16-bit, the game stays silent rather than playing garbage.
- **Mixer fallback init.** If pygame.init() didn't start the mixer, _init_audio starts it with SAMPLE_RATE.
- **Tuning constants.** BOUNCE_COOLDOWN_MS, MIN_BOUNCE_SPEED and SAMPLE_RATE are named at the top so you can adjust them easily.
- **self._last_bounce_ticks = -BOUNCE_COOLDOWN_MS.** This makes the first bounce of the game play immediately.

I also left Marble.rect() in place, even though the engine no longer uses it after Task 1, so marble.py and wall.py stayed unchanged.

If any of this conflicts with your lab's rubric, for example if it must use exactly two states or shouldn't add new methods, tell me which parts to remove and I'll send a trimmed file.